Kryptografia - Pentest

POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

Spis treści

Wstęp 3

Przygotowanie środowiska 4

Uruchamianie maszyn 5

Połączenie OpenVPN 6

Dodatkowe uwagi 7

Zadania 8

Wprowadzenie do Hashcat 9

Zadanie 9.1p: Hash md5 10

Zadanie 9.2p: Hash sha1 11

Zadanie 9.3p: Hash bcrypt 12

Zadanie 9.4p: Hash NTLM 13

Zadanie 9.5p: Archiwum ZIP 15

Zadanie 9.6p: Archiwum RAR 17

Referencje 22

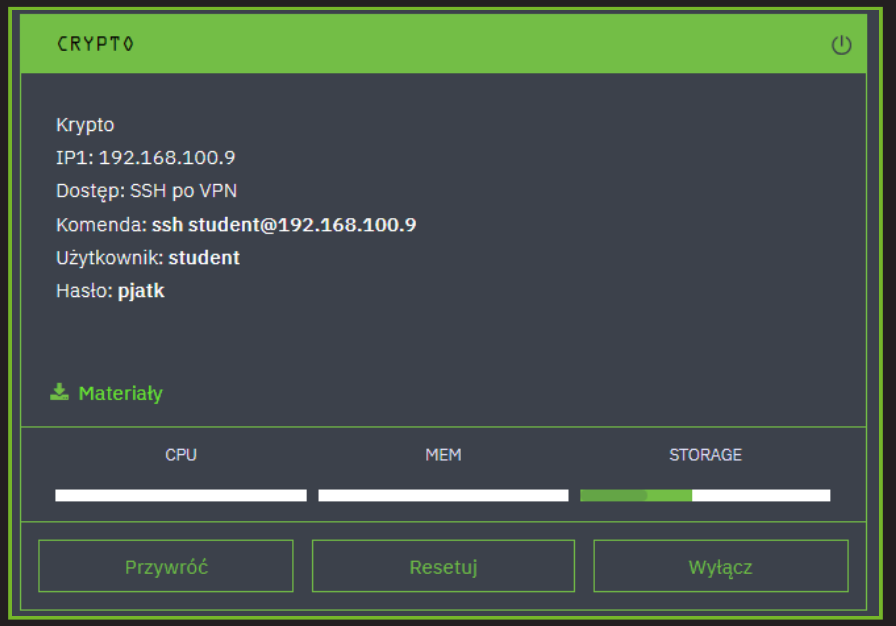
Wstęp

Student pozna programy Hashcat i John the Ripper, z pomocą których złamie hasła zapisane w formie hashy oraz uzyska dostęp do zaszyfrowanych archiwów.

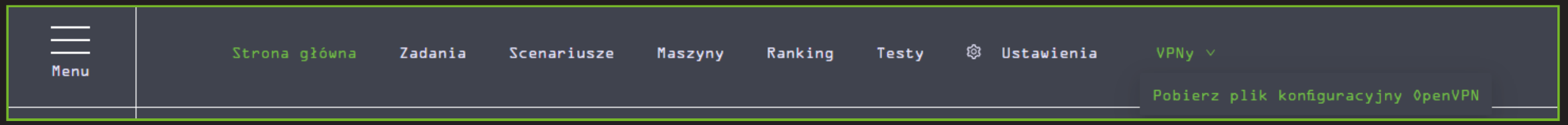
Przygotowanie środowiska

Uruchamianie maszyn

Przed rozpoczęciem upewnij się, że maszyna Crypto jest włączona.
Jeżeli coś się zepsuje, w każdej chwili możesz ją przywrócić do stanu początkowego.



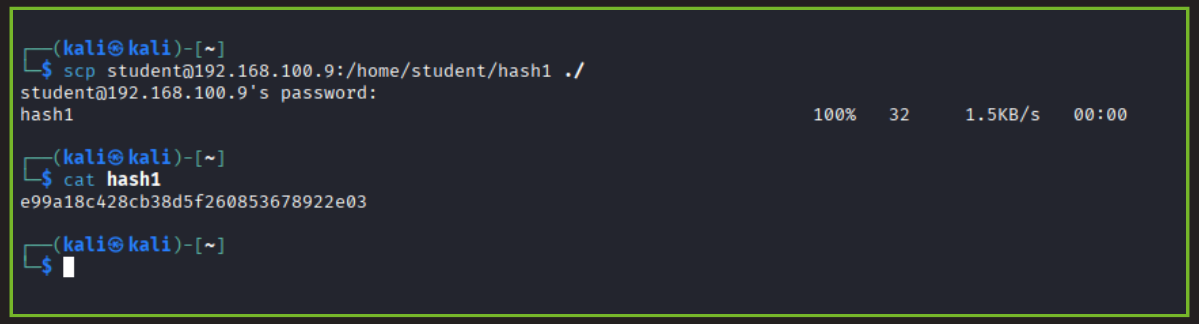
Połączenie OpenVPN



Połączenie OpenVPN jest niezbędne do rozwiązania zadań.
Aby połączyć się z siecią VPN, należy pobrać plik konfiguracyjny OpenVPN z rozwijanego menu „**VPNy**” na stronie głównej.
Następnie należy wykonać komendę:

```
sudo openvpn /sciezka/do/pliku.ovpn
```

Dodatkowe uwagi



- Wszystkie zadania z tego dokumentu wykonujemy na maszynie Kali.
- Flagi w niektórych zadaniach mają format inny od standardowego!
- Plik do zadania możesz pobrać na swoją maszynę za pomocą komendy:
  `scp student\@192.168.100.9:/home/student/{nazwapliku} ./`
- W używanej wersji systemu słownik rockyou.txt jest już wypakowany. Domyślnie na Kali możesz potrzebować zrobić to samodzielnie:
  `sudo gunzip /usr/share/wordlists/rockyou.txt.gz`

Zadania

Wprowadzenie do Hashcat

Hashcat to zaawansowane narzędzie do **odzyskiwania haseł**, wykorzystywane głównie w testach penetracyjnych i analizach bezpieczeństwa. Działa z różnorodnymi formatami skrótów (hashy) i potrafi efektywnie wykorzystać moc obliczeniową **procesorów graficznych** (GPU), znacząco przyspieszając proces łamania haseł. Dzięki temu stanowi istotne wsparcie dla specjalistów ds. bezpieczeństwa, którzy potrzebują szybkich i skalowalnych rozwiązań. Oprogramowanie jest dostępne na zasadach open source, co sprzyja jego nieustannemu rozwojowi i ulepszaniu metod ataku. Hashcat wyróżnia się elastycznością w dostosowywaniu parametrów pracy, umożliwiając użytkownikowi precyzyjne kontrolowanie procesu łamania. Dodatkowo, aktywna społeczność oraz regularne aktualizacje zapewniają, że narzędzie pozostaje na bieżąco z nowymi technikami szyfrowania i rosnącymi potrzebami w zakresie testów bezpieczeństwa.

Zadanie 9.1p: Hash md5

W katalogu domowym znajduje się plik hash1, który zawiera hash MD5. Użyj programu hashcat, aby poznać zaszyfrowany ciąg znaków. Użyto ciągu, który znajduje się w słowniku rockyou.txt.

```
hashcat -m 0 hash1 /usr/share/wordlists/rockyou.txt
```

```
└─$ hashcat -m 0 hash1 /usr/share/wordlists/rockyou.txt
hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian  Linux, None+Asserts, RELOC, LLVM 17.0.6, SLEEF, DISTRO, POCL_DEBUG)
 - Platform #1 [The pocl project]
═══════════════════════════════════════════════════════════════════════════════
* Device #1: cpu-sandybridge-11th Gen Intel(R) Core(TM) i9-11900KF @ 3.50GHz, 2913/5891 MB (1024 MB allocat
able), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0×0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Hash
* Single-Salt
* Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.
```

Hashe MD5 były popularnie stosowane do zapisywania haseł w bazach danych. Obecnie odchodzi się od nich, jednak nadal bywają używane zarówno w aplikacjach internetowych, jak i działających lokalnie.

Zadanie 9.2p: Hash sha1

W katalogu domowym znajduje się plik hash2, który zawiera hash SHA1. Użyj programu hashcat, aby poznać zaszyfrowany ciąg znaków. Użyto ciągu, który znajduje w słowniku rockyou.txt.

```
hashcat -m 100 hash2 /usr/share/wordlists/rockyou.txt
```

```
┌──$ hashcat -m 100 hash2 /usr/share/wordlists/rockyou.txt
hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian  Linux, None+Asserts, RELOC, LLVM 17.0.6, SLEEF, DISTRO, POCL_DEBUG)
 - Platform #1 [The pocl project]
=============================================================================

* Device #1: cpu-sandybridge-11th Gen Intel(R) Core(TM) i9-11900KF @ 3.50GHz, 2913/5891 MB (1024 MB allocat
able), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0×0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Hash
* Single-Salt
* Raw-Hash
```

Hashe SHA1 uchodzą za bezpieczniejsze od MD5 i nadal są często stosowane w bazach danych dużych serwisów internetowych do zapisywania haseł użytkowników. Nierzadko stosowane jest podwójne hashowanie: *sha1(sha1(hasło))*.

Zadanie 9.3p: Hash bcrypt

W katalogu domowym znajduje się plik hash3, który zawiera hash bcrypt. Użyj programu hashcat, aby poznać zaszyfrowany ciąg znaków. Użyto ciągu, który znajduje w słowniku rockyou.txt.

```
hashcat -m 3200 hash3 /usr/share/wordlists/rockyou.txt
```

```
└─$ hashcat -m 3200 hash3 /usr/share/wordlists/rockyou.txt
hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian  Linux, None+Asserts, RELOC, LLVM 17.0.6, SLEEF, DISTRO, POCL_DEBUG)
 - Platform #1 [The pocl project]
═══════════════════════════════════════════════════════════════════════════════
* Device #1: cpu-sandybridge-11th Gen Intel(R) Core(TM) i9-11900KF @ 3.50GHz, 2913/5891 MB (1024 MB allocat
able), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 72

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0×0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Single-Hash
* Single-Salt

Watchdog: Temperature abort trigger set to 90c

Host memory required for this attack: 0 MB

Dictionary cache hit:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344385
* Bytes.....: 139921507
* Keyspace..: 14344385
```

Bcrypt jest coraz popularniejszym algorytmem hashującym ze względu na znaczną poprawę bezpieczeństwa poprzez umożliwienie dodania do szyfrowanego ciągu znaków tzw. „soli” - dodatkowych znaków. Jego obliczenie jest jednak dłuższe niż w przypadku MD5 i SHA1.

W hashu bcrypt występuje znak $, który może być różnie interpretowany w konsoli. Uważaj na to.

Zadanie 9.4p: Hash NTLM

W katalogu domowym znajduje się plik hash4, który zawiera hash NTLM. Użyj programu hashcat, aby poznać zaszyfrowany ciąg znaków. Tym razem użyto hasła składającego się z jedenastu cyfr, które nie znajduje się w słowniku

```
hashcat -m 1000 hash4 -a3 ?d?d?d?d?d?d?d?d?d?d?d
```

```
└─$ hashcat -m 1000 hash4 -a3 ?d?d?d?d?d?d?d?d?d?d?d
hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian  Linux, None+Asserts, RELOC, LLVM 17.0.6, SLEEF, DISTRO, POCL_DEBUG)
 - Platform #1 [The pocl project]
====================================================================================================
* Device #1: cpu-sandybridge-11th Gen Intel(R) Core(TM) i9-11900KF @ 3.50GHz, 2913/5891 MB (1024 MB allocat
able), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0×0000ffff mask, 262144 bytes, 5/13 rotates

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Hash
* Single-Salt
* Brute-Force
* Raw-Hash
```

NTLM to metoda hashowania opracowana przez Microsoft i stosowana do zapisu haseł użytkowników w systemach z rodziny NT (m.in. Windows XP).

Uwaga! Złamanie tego hasła może zająć kilka minut. Jeśli szacowany czas wykonania wynosi kilka godzin, oznacza to błędne podejście. Należy ponownie sprawdzić poprawność polecenia.

Zadanie 9.4p: Hash NTLM

Uwaga! Hashcat ma wbudowany mechanizm, który sprawia, że jeśli już kiedyś złamał dane hasło, nie będzie próbował ponownie i nie wyświetli wyniku! W takiej sytuacji należy dodać na końcu polecenia przełącznik `--show`. Alternatywnie można wyświetlić zawartość pliku .pot (np. za pomocą polecenia `cat`).

Zadanie 9.5p: Archiwum ZIP

W katalogu domowym znajduje się plik pkzip.zip, który zawiera plik z flagą. Użyjemy programu John the Ripper do złamania hasła. Najpierw musimy jednak przygotować plik wejściowy na podstawie naszego archiwum. Hasło do archiwum znajduje się w słowniku rockyou.txt.

```
zip2john pkzip.zip > hash5
john --wordlist=/usr/share/wordlists/rockyou.txt hash5
unzip pkzip.zip
```

Zadanie 9.5p: Archiwum ZIP

```
┌──(kali㉿kali)-[~]
└─$ zip2john pkzip.zip > hash5

ver 1.0 efh 5455 efh 7875 pkzip.zip/flag.txt PKZIP Encr: 2b chk, TS_chk, cmplen=37, decmplen=25, crc=16FE0F
6E ts=4D2C cs=4d2c type=0

┌──(kali㉿kali)-[~]
└─$ john --wordlist=/usr/share/wordlists/rockyou.txt hash5
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
[redacted]       (pkzip.zip/flag.txt)
1g 0:00:00:00 DONE (2024-08-15 13:09) 2.173g/s 23186Kp/s 23186Kc/s 23186KC/s SANOMI..Ryan0330
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

┌──(kali㉿kali)-[~]
└─$ unzip pkzip.zip
Archive:  pkzip.zip
[pkzip.zip] flag.txt password:
 extracting: flag.txt

┌──(kali㉿kali)-[~]
└─$ cat flag.txt
p[redacted]
```

Zadanie 9.6p: Archiwum RAR

W katalogu domowym znajduje się file.rar, który zawiera nieznany plik. Ponownie użyjemy programu John the Ripper do złamania hasła. Najpierw musimy jednak przygotować plik wejściowy na podstawie naszego archiwum. Hasło do archiwum nie znajduje się w słowniku. Wiemy jednak, że składa się z małych liter oraz cyfr i ma maksymalnie trzy znaki.
Użyjemy więc programu crunch, aby stworzyć słownik właśnie takich haseł:

```
crunch 1 3 -f /usr/share/crunch/charset.lst lalpha-numeric > new_dict.txt
```

```
└─$ crunch 1 3 -f /usr/share/crunch/charset.lst lalpha-numeric > new_dict.txt
Crunch will now generate the following amount of data: 190584 bytes
0 MB
0 GB
0 TB
0 PB
Crunch will now generate the following number of lines: 47988
```

Zadanie 9.6p: Archiwum RAR

Następnie musimy utworzyć plik wejściowy dla John the Ripper na podstawie naszego archiwum, a później uruchomić łamanie hasła z utworzonym słownikiem. Uwaga, łamanie hasła może potrwać kilka minut.

```
rar2john file.rar > hash6
john --wordlist=new_dict.txt hash6
```

```
┌──(kali㉿kali)-[~]
└─$ rar2john file.rar > hash6


┌──(kali㉿kali)-[~]
└─$ john --wordlist=new_dict.txt hash6

Using default input encoding: UTF-8
Loaded 1 password hash (RAR5 [PBKDF2-SHA256 128/128 AVX 4x])
Cost 1 (iteration count) is 32768 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
[redacted]              (file.rar)
1g 0:00:00:37 DONE (2024-08-15 13:15) 0.02642g/s 781.1p/s 781.1c/s 781.1C/s v0u..v2l
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

Zadanie 9.6p: Archiwum RAR

Spróbujmy wypakować archiwum i wyświetlić zawartość pliku. Czy wyświetla się poprawnie?

```
unrar x file.rar cat file
```

```
└─$ unrar x file.rar

UNRAR 7.01 freeware      Copyright (c) 1993-2024 Alexander Roshal


Extracting from file.rar

Enter password (will not be echoed) for file:

Extracting  file                                                      OK
All OK

┌──(kali㉿kali)-[~]
└─$ cat file
@@@@◆▓▓▓PP   ◆◆◆-◆=◆=PX◆-◆=◆=◆888 XXXDDS◆td888 P◆td▓ ▓ ▓ ,,Q◆tdR◆td◆-◆=◆=00/lib64/ld-linux-x86-64.so.2GNU◆◆
GNU◆l1.x8i◆R◆◆◆◆ᵹ◆◆VGNU◆◆e◆mQ ")m | "__cxa_finalize__libc_start_mainstrlenprintflibc.so.6GLIBC_2.2.5GLIBC_2
.34_ITM_deregisterTMCloneTable__gmon_start___ITM_registerTMCloneTable0u▓i       :◆◆◆F◆@◆▓▓@◆?◆?◆?◆?@H◆H◆◆
/H◆◆t◆◆H◆◆◆5◆/◆%◆/@◆%◆/h◆◆◆◆◆◆%◆/h◆◆◆◆◆◆%◆/f◆1◆I◆◆^H◆◆H◆◆◆PTE1◆1◆H◆=◆◆?/◆f.◆@H◆=◆/H◆◆/H9◆tH◆/H◆◆t       ◆◆◆
◆◆H◆=Y/H◆5R/H)◆H◆◆H◆◆?H◆◆H◆H◆◆tH◆◆.H◆◆◆◆fD◆◆◆◆◆=/u+UH◆=◆.H◆◆t
                                                                H◆=◆.◆)◆◆◆◆d◆◆◆◆◆.]◆◆◆◆◆◆w◆◆◆UH◆◆SH◆◆XH◆flag:b
1nH◆@Ry_f1L3H◆E◆H◆U◆◆E◆H◆46567hdmH◆mmd33442H◆E◆H◆U◆◆E◆◆E◆◆◆*◆E◆H◆◆ThE◆H◆◆D◆1JE◆◆E◆H◆◆U◆◆T◆◆E◆◆E◆Hc◆H◆E◆H◆◆◆Y
◆◆◆H9◆r◆◆E◆H◆◆D◆H◆H◆N◆F◆◆◆◆H◆]◆◆◆H◆H◆◆Wykonano/Executed!◆◆◆x8◆◆◆◆H◆◆◆H1◆◆◆◆zRx
                                                                      ◆◆◆◆"zRx
                                                                            $◆◆◆◆0F▓J
                                                                                  ◆?▓;*3$"D◆◆◆ \
```

Zadanie 9.6p: Archiwum RAR

Taki widok świadczy o tym, że możemy mieć do czynienia z plikiem binarnym. Uruchamianie nieznanego pliku z zaszyfrowanego archiwum zwykle nie jest dobrym pomysłem, jednak tym razem nie jest to wirus. Zobaczmy, co stanie się po uruchomieniu.

```
./file
```



Zadanie 9.6p: Archiwum RAR

Mamy więc już pewność, że jest to binarny plik wykonywalny. W jaki sposób możemy poszukać w nim flagi? Mamy dwie możliwości, jednak obie będą wymagały od nas odpowiedniej analizy otrzymanych wyników. Wyszukajmy stringi lub otwórzmy plik w odpowiedniej aplikacji:

`strings file` lub `xxd file`

Znalezienie poprawnej flagi pozostawiamy studentom!

```
└─$ strings file
/lib64/ld-linux-x86-64.so.2
1.x8i
__cxa_finalize
__libc_start_main
strlen
printf
libc.so.6
GLIBC_2.2.5
GLIBC_2.34
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
PTE1
u+UH
```

```
└─$ xxd file
00000000: 7f45 4c46 0201 0100 0000 0000 0000 0000  .ELF............
00000010: 0300 3e00 0100 0000 6010 0000 0000 0000  ..>.....`.......
00000020: 4000 0000 0000 0000 c036 0000 0000 0000  @........6......
00000030: 0000 0000 4000 3800 0d00 4000 1f00 1e00  ....@.8...@.....
00000040: 0600 0000 0400 0000 4000 0000 0000 0000  ........@.......
00000050: 4000 0000 0000 0000 4000 0000 0000 0000  @.......@.......
00000060: d802 0000 0000 0000 d802 0000 0000 0000  ................
00000070: 0800 0000 0000 0000 0300 0000 0400 0000  ................
00000080: 1803 0000 0000 0000 1803 0000 0000 0000  ................
00000090: 1803 0000 0000 0000 1c00 0000 0000 0000  ................
000000a0: 1c00 0000 0000 0000 0100 0000 0000 0000  ................
000000b0: 0100 0000 0400 0000 0000 0000 0000 0000  ................
000000c0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000d0: 5006 0000 0000 0000 5006 0000 0000 0000  P.......P.......
000000e0: 0010 0000 0000 0000 0100 0000 0500 0000  ................
000000f0: 0010 0000 0000 0000 0010 0000 0000 0000  ................
00000100: 0010 0000 0000 0000 1102 0000 0000 0000  ................
00000110: 1102 0000 0000 0000 0010 0000 0000 0000  ................
00000120: 0100 0000 0400 0000 0020 0000 0000 0000  ..... ..... ....
00000130: 0020 0000 0000 0000 0020 0000 0000 0000  . ..... ..... ....
00000140: f800 0000 0000 0000 f800 0000 0000 0000  ................
00000150: 0010 0000 0000 0000 0100 0000 0600 0000  ................
00000160: d02d 0000 0000 0000 d03d 0000 0000 0000  .-.......=......
00000170: d03d 0000 0000 0000 5002 0000 0000 0000  .=......P.......
00000180: 5802 0000 0000 0000 0010 0000 0000 0000  X...............
00000190: 0200 0000 0600 0000 e02d 0000 0000 0000  .....-..........
000001a0: e03d 0000 0000 0000 e03d 0000 0000 0000  .=.......=......
000001b0: e001 0000 0000 0000 e001 0000 0000 0000  ................
```

Referencje

- Struktura plików ZIP: https://users.cs.jmu.edu/buchhofp/forensics/formats/pkzip.html
- Struktura plików RAR: https://ctf-wiki.mahaloz.re/misc/archive/rar/
- Podstawy uwierzytelniania NTLM: https://tinyurl.com/aboutntlm
- Przełączniki programu Hashcat: https://hashcat.net/wiki/doku.php?id=example_hashes